COMP6713 Sentiment Analysis

2026 Term 1

Group name:
Never Fail and I love NLP

Qinglin Tan	z5579853
Zixu Cheng	z5628034
Yanyan Zhu	z5565492
Tianji Liu	z5613025
Zekai Gao	z5548102

UNSW SYDNEY

Project Introduction

Binary sentiment analysis comparing four models: SentiWordNet baseline, zero-shot DistilBERT, zero-shot RoBERTa, and fine-tuned DistilBERT. Includes a Gradio web demo.

Data Pipeline

1.Contribution:

Built data preprocessing and pipeline

Converted raw datasets into standardized

formatOutput: train.json / val.json / test.json

Conducted domain analysis (length, vocabulary, OOV, style)

Generated dataset statistics and analysis reports

2.Data Sources:

GoEmotions → training & validation

IMDb → testing (cross-domain)

3.Data Pipeline Design:

Modular pipeline:

label_mapping.py → convert labels

clean.py → text preprocessing

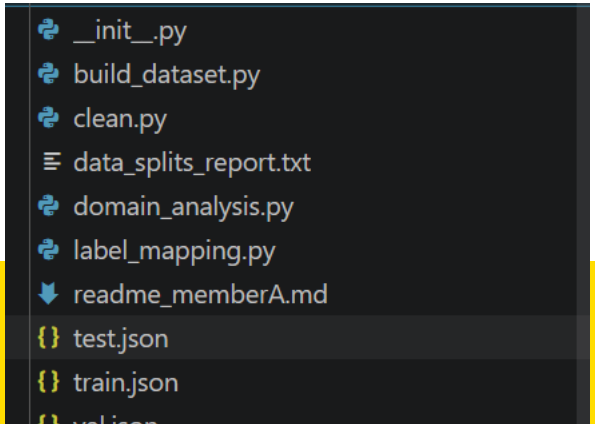
build_dataset.py → dataset construction

4.Data Processing Steps:

Label mapping (27 emotions → binary sentiment)

Text cleaning (remove noise, normalize text)

Dataset splitting (train / val / test)



Dataset Summary

- Btrain.json: 20,484 samples

positive: 13,270

negative: 7,214

val.json: 5,121 samples

positive: 3,318

negative: 1,803

test.json: 25,000 samples

positive: 12,500

negative: 12,500

Average text length:

Train: 12.8 tokens

Test: 224.5 tokens

Length range:

Train: 1–31 tokens

Test: 4–2192 tokens

data > data_splits_report.txt

```
summary of processed datasets (GoEmotions train,val, IMDb test)

train.json: 20484 samples (pos: 13270 / neg: 7214)
val.json: 5121 samples (pos: 3318 / neg: 1803)
test.json: 25000 samples (pos: 12500 / neg: 12500)
avg text length: train=12.8 tokens, test=224.5 tokens
shortest and longest texts in each split:
train length range: 1 - 31 tokens
test length range: 4 - 2192 tokens
```

Domain analysis

- analysis:

length analysis | vocab overlap | OOV | style analysis | domain summary

- 1. Length Difference

Train: ~13 tokens Test: ~233 tokens ≈17× longer

```
train length statistics
count: 20484
min: 1
max: 31
mean: 13.33
median: 13.00
std: 6.91
q1: 8.00
q3: 19.00
p90: 23.00
```

```
test length statistics
count: 25000
min: 6
max: 2303
mean: 232.72
median: 175.00
std: 171.68
q1: 128.00
q3: 282.00
p90: 452.00
```

- 2. Vocabulary Gap

Jaccard overlap: 0.17 (low) OOV words: 81% unique

```
vocab_jaccard_overlap: 0.172250
oov_unique_ratio: 0.819729
```

- 3. Writing Style Difference

Train: short, informal

Test: long, structured reviews

```
train text style statistics
avg_char_length: 68.12
avg_sentence_count: 1.49
avg_exclamation_count: 0.24
```

```
test text style statistics
avg_char_length: 1269.59
avg_sentence_count: 13.03
avg_exclamation_count: 0.98
```

- 4.Domain Shift Impact

Large difference in: length | vocabulary | style

```
domain shift summary
test_vs_train_mean_length_ratio: 17.462208
test_vs_train_unique_token_ratio: 4.408540
vocab_jaccard_overlap: 0.172250
test_oov_token_ratio_against_train: 0.083180
test_oov_unique_ratio_against_train: 0.819729
test_vs_train_sentence_count_ratio: 8.744807
test_vs_train_punctuation_ratio: 0.644189
```

Model Implementation Details

- 1. NLTK Baseline (SentiWordNet)

(1)Role: Implemented the lexicon-based baseline.

(2)Method: Used sentiment scores from SentiWordNet for rule-based inference.

(3)Purpose: Established a performance benchmark for traditional NLP methods.

baseline.py

pretrained...

pretrained...

2. RoBERTa

(1)Role: Led the fine-tuning process.

(2)Setup: Configured Hugging Face Transformers, optimized hyperparameters, and trained on the target dataset.

3.DistilBERT

(1)Role: Conducted comparative analysis of efficient alternatives.

(2)Approach: Implemented knowledge distillation for speed/accuracy trade-off.

(3)Key Objective: Verify if a smaller model can match or exceed RoBERTa.

baseline_p...

pretrained...

Model Implementation Details

1. NLTK Baseline (SentiWordNet)

(1)Tokenization & POS Tagging: Split text into words and tag their parts of speech

(2)POS Mapping: Convert tags to WordNet-compatible format for SentiWordNet queries.

(3)Sentiment Score Lookup: Retrieve pre-defined positive/negative scores for each valid word.

```python
def get_sentiment_score(text: str) -> int:
    # Tokenize text and perform POS tagging
    tokens = word_tokenize(text)
    tagged = pos_tag(tokens)

    pos_score, neg_score = 0.0, 0.0
    for token, tag in tagged:
        wn_tag = get_wordnet_pos(tag)
        if wn_tag is None:
            continue
        # Get sentiment scores from SentiWordNet
        synsets = list(swn.senti_synsets(token, pos=wn_tag))
        if synsets:
            # Use the first synset score (baseline simplification)
            best = synsets[0]
            pos_score += best.pos_score()
            neg_score += best.neg_score()
```

2. RoBERTa

(1) Batch Inference Loop

(2)Model Inference: Call the fine-tuned RoBERTa classifier for sentiment analysis

(3)Structured Logging

```python
def predict_roberta(data: list) -> list:
    predictions = []
    for idx, item in enumerate(data):
        result = classifier(item['text'])[0]
        pred = LABEL_MAP[result['label']]
        predictions.append({
            "id": idx,
            "text": item['text'],
            "true_label": item['label'],
            "predicted_label": pred,
            "confidence": round(result['score'], 4),
            "model": "roberta-base-sentiment"
        })
    return predictions
```

3.DistilBERT

(1)Lightweight Model Inference

(2)Unified Label Mapping

(3)Standardized Output Logging

```python
def predict_distilbert(data: list) -> list:
    predictions = []
    for idx, item in enumerate(data):
        result = classifier(item['text'])[0]
        pred = LABEL_MAP[result['label']]
        predictions.append({
            "id": idx,
            "text": item['text'],
            "true_label": item['label'],
            "predicted_label": pred,
            "confidence": round(result['score'], 4),
            "model": "distilbert-sst-2"
        })
    return predictions
```

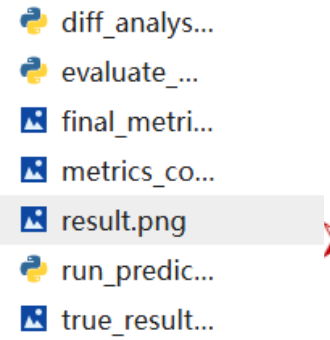
Comparative Evaluation & Key Insights

1. Data details

(1)49.96% (All 3 Agree): Clear consensus cases

(2)35.22% :Primary source of error for the traditional method.

(3)14.82%: Complex cases where deep models diverge.



2. Key Takeaways:

(1)Pre-trained models (RoBERTa/DistilBERT) significantly outperform the NLTK baseline.

(2)DistilBERT offers an optimal balance of speed and accuracy for real-world applications.

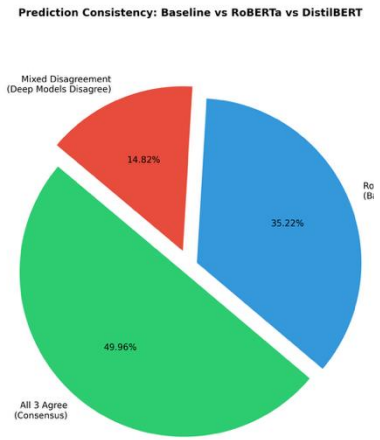
3. Visual 1:Performance Metrics

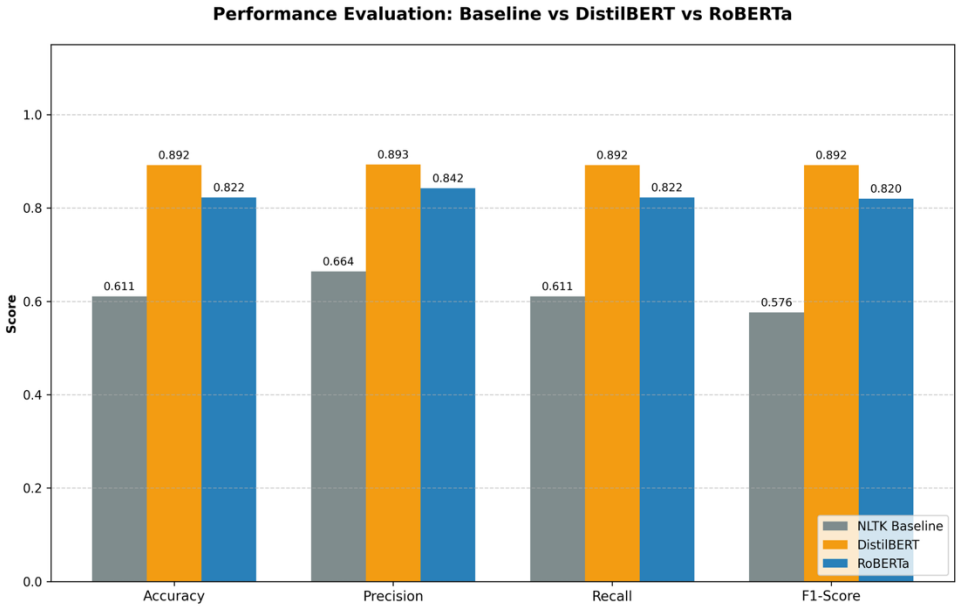
(1)Metrics: Accuracy, Precision, Recall, F1-Score.

(2)Comparison: NLTK Baseline vs. RoBERTa vs. DistilBERT.

(3)Highlight: DistilBERT outperforms RoBERTa across all metrics

4.Visual 2:Prediction Consistency





DistilBERT Fine-tuning

1.Model Setup:

Base model: distilbert-base-uncased

Max sequence length: 128 tokens

Labels: binary (negative / positive)

Platform: Google Colab (T4 GPU)

2.Training Configuration:

Epochs: 3 | Batch size: 32

Learning rate: 2e-5 | Weight decay: 0.01

Eval strategy: per epoch | Metric: F1 (binary)

3.In-domain Results (GoEmotions val set):

Validation F1: 0.928 | Accuracy: 0.906

Tracking: Weights & Biases (W&B)

W&B Sweep

1.Sweep Design:

Method: grid search (exhaustive)

learning_rate: {2e-5, 3e-5}

num_epochs: {2, 3} → 4 runs total

2.Sweep Results:

Run 1: lr=2e-5, ep=2 → F1=0.9249, Acc=0.9028

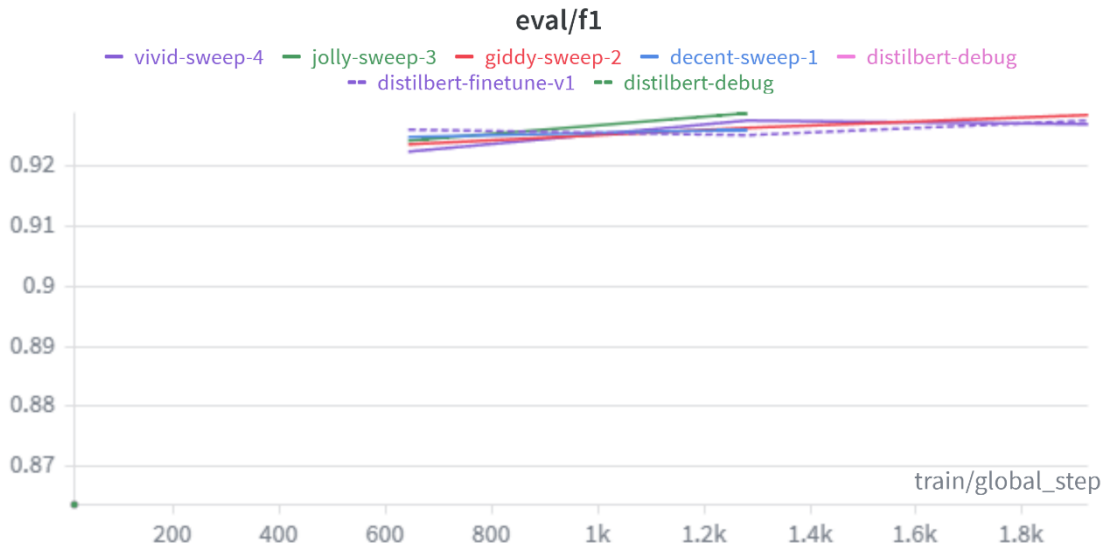
Run 2: lr=2e-5, ep=3 → F1=0.9286, Acc=0.9072

Run 3 (best): lr=3e-5, ep=2 → F1=0.9290

Run 4: lr=3e-5, ep=3 → F1=0.9272

3.Best Config:

lr=3e-5, epochs=2 (3 epochs showed overfitting)



Model Analysis

1.Sweep Summary:

All 4 runs achieved F1 > 0.924 on GoEmotions val

lr=3e-5 outperforms lr=2e-5 across both epoch counts

epoch=3 shows slight overfitting vs epoch=2

2.FinetunedPredictor Interface:

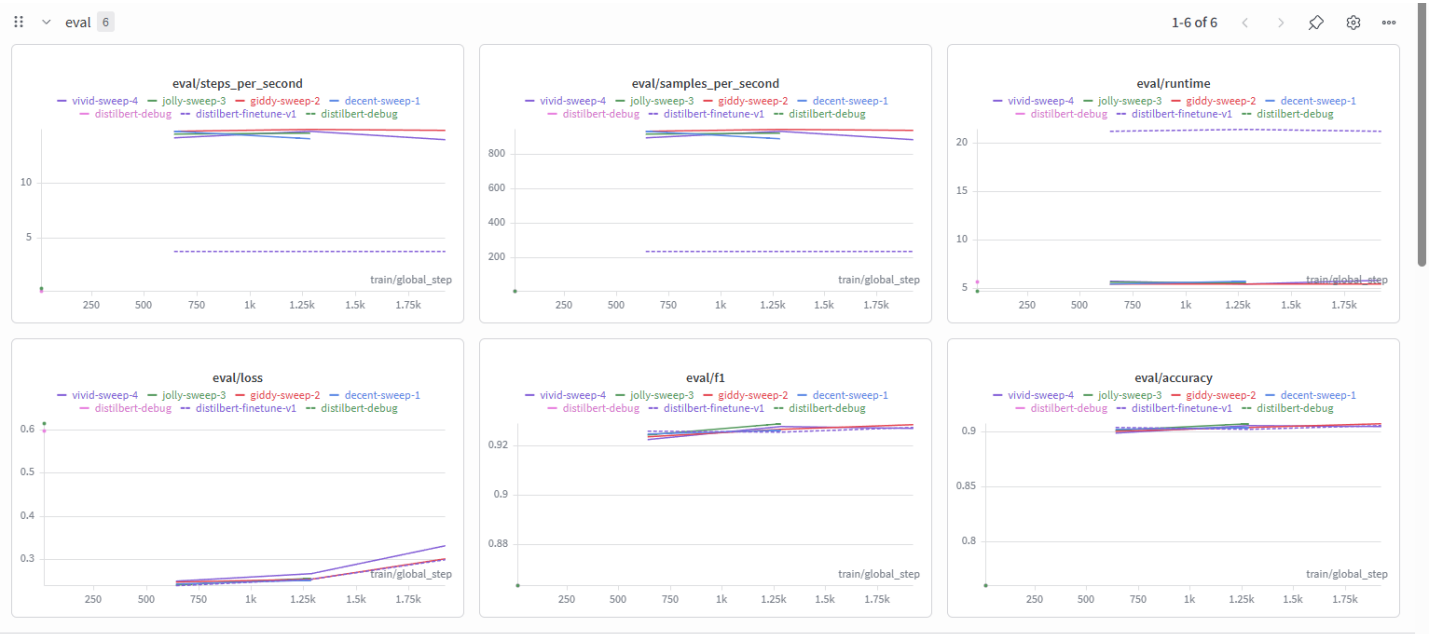
Unified inference class for evaluation and demo.

Returns: label, confidence, label_text

3.Model Saved To:

GitHub (Git LFS) + Google Drive backup

models/distilbert-finetuned/best_model/



RoBERTa Fine-tuning via W&B Hyperparameter Sweep

Setup:

Platform: RTX 4090 24GB

Base model: roberta-base

Max sequence length: 128 tokens

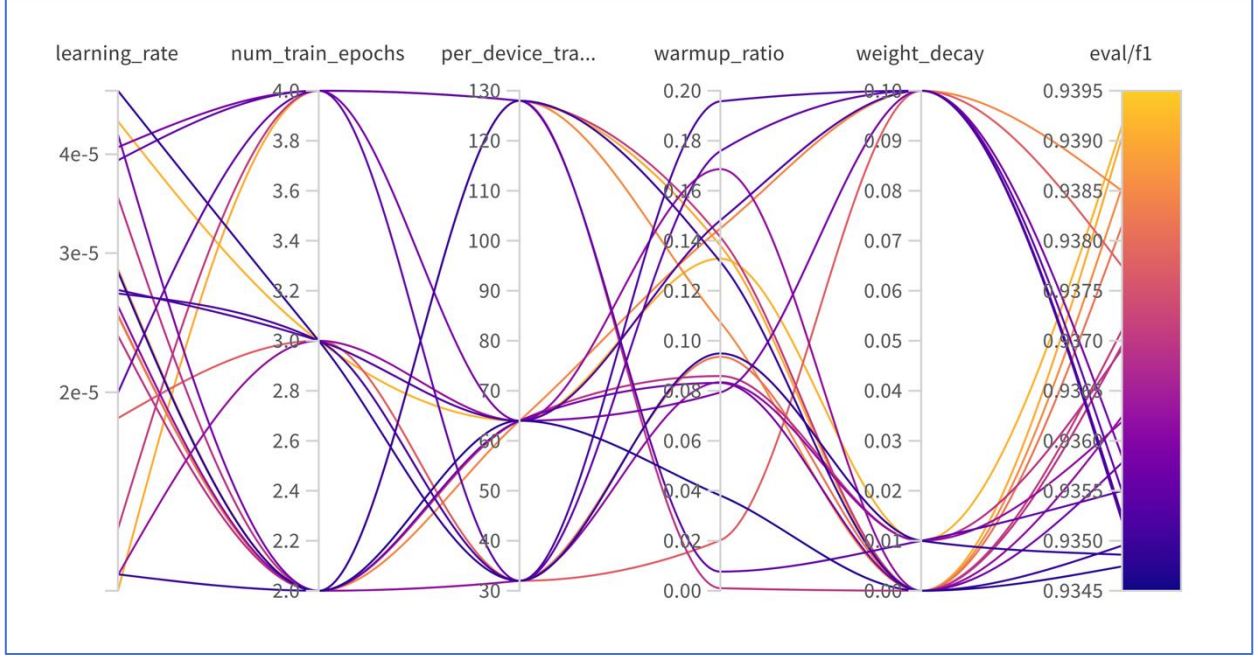
Labels: binary (negative / positive)

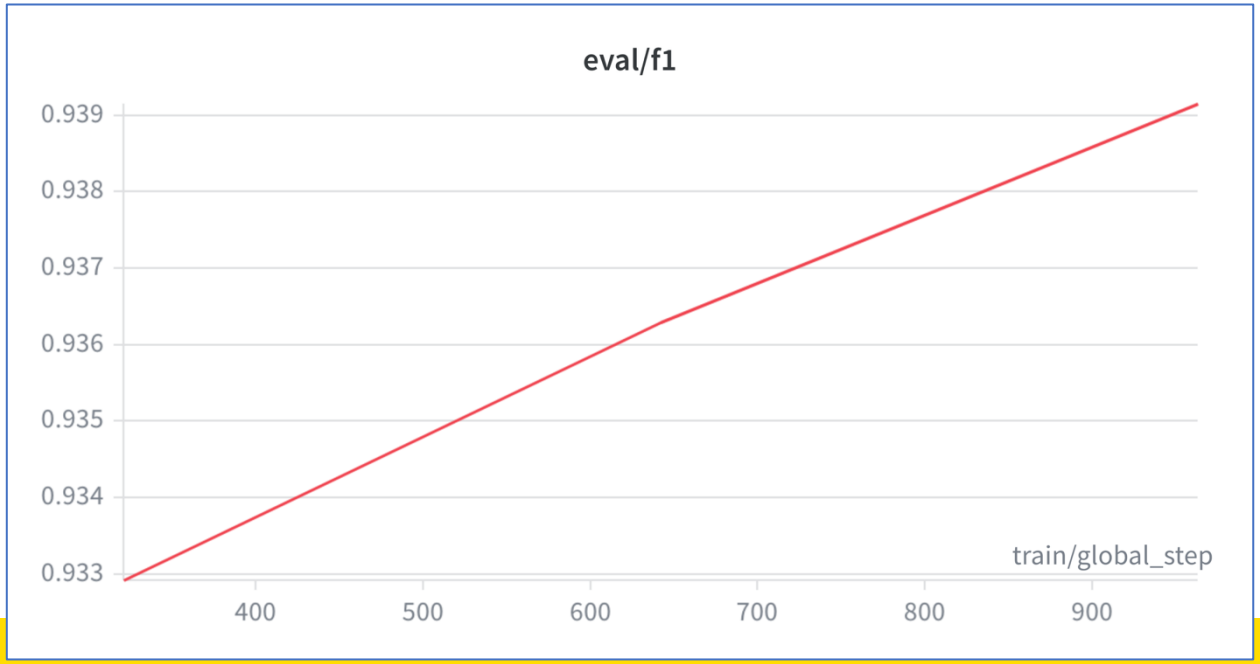
Search method: Bayesian optimization (20 trials)

Search space: lr, batch_size, epochs, weight_decay, warmup_ratio

Best config: lr=4.08e-5, epochs=4, batch_size=64, weigh_decay=0.1, warmup_ratio=0.079

Best Validation F1 = 0.93915





CLI (predict.py)

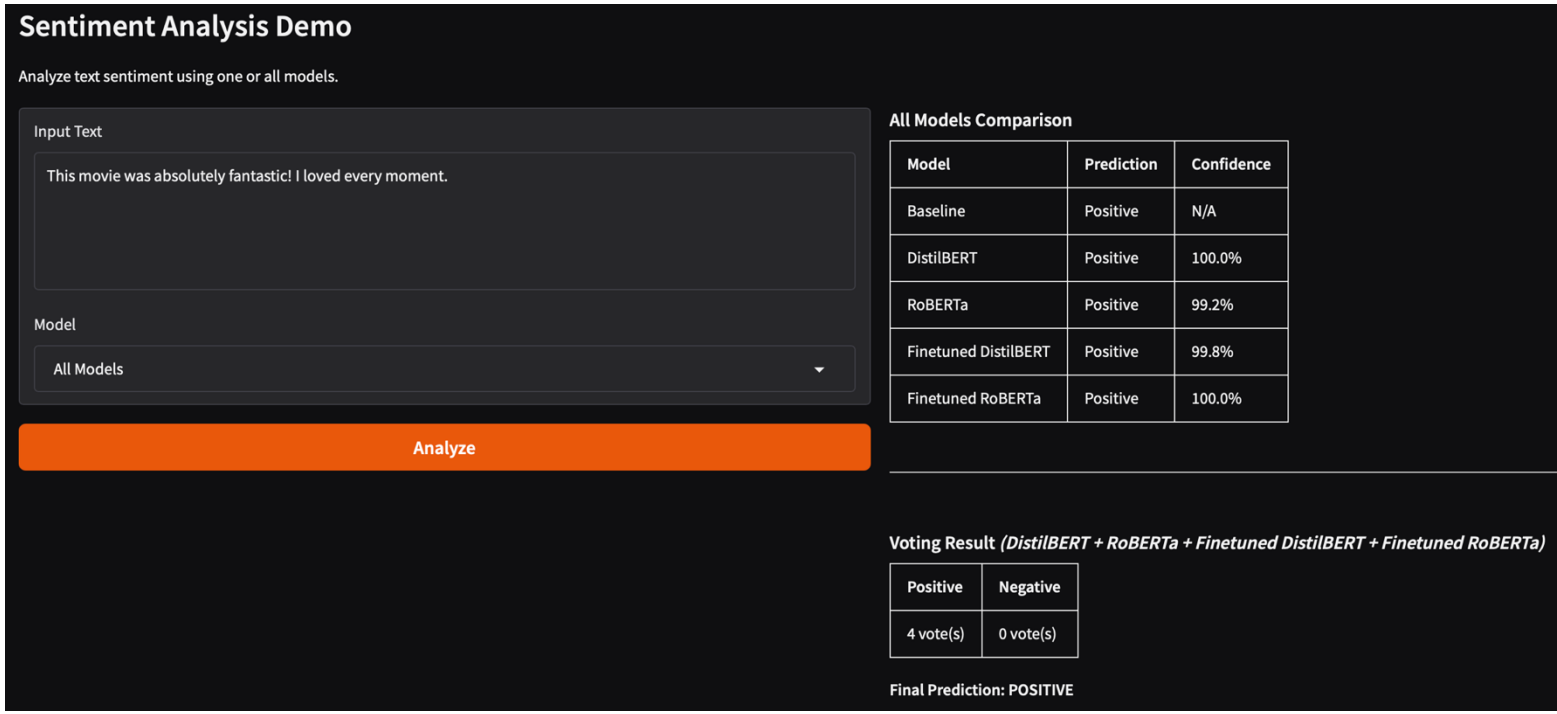
- 5 models supported
- '--text' / '--file' input
- Return lable + confidence

```
(6713_ass) zekai_gao@DESKTOP-8254V9I:~/Programs/comp6713-sentiment-analysis$ python demo/predict.py
 --text "The service was terrible and I will never come back." --model finetuned-roberta

____________________________________________________
Text: The service was terrible and I will never come back.
Predicted Label: 0
Label Text: negative
Confidence: 0.9981
```

Gradio (app.py)

- Web UI, 6 model options
- Single model or **ALL MODELS**
- Comparison table + **majority voting**



Quantitative Results

Model	Accuracy	Precision	Recall	F1
SentiWordNet Baseline	0.6107	0.5704	0.8964	0.6972
RoBERTa (zero-shot)	0.8223	0.9242	0.7022	0.7981
DistilBERT-SST2 (zero-shot)	0.8921	0.9151	0.8644	0.8890
DistilBERT (fine-tuned)	0.7523	0.7197	0.8266	0.7695
RoBERTa (fine-tuned)	0.7744	0.7220	0.8925	0.7983

Error Analysis

1. Negation
2. Sarcasm
3. Domain Mismatch
4. Ambiguous
5. Other

Example

```
"true_label": 0,
"predicted_label": 1,
"confidence": 0.5654,
```

This film is so bad it's hilarious... It is very amusing though, and I guess either way it could go down as a cult classic, either from being an amusing parody of the Tarantino/Rodriguez style, or something that is very very unintentionally funny. Has to be seen to be believed.

Ambigous

Error Analysis

Error Category	DistilBERT (n = 30)	RoBERTa (n = 30)	Combined (n = 60)	Combined%
Negation	3	4	7	12%
Sarcasm	6	4	10	17%
Domain Mismatch	4	7	11	18%
Ambiguous	11	8	19	32%
Other	6	7	13	22%
Total	30	30	60	100%

Table 2 Distribution of error categories across 60 manually analysed misclassified samples

Discussion and Conclusion

- **Best model:** DistilBERT-SST2 (zero-shot)
  Accuracy = **89.2%**, F1 = **0.889**
- **Main finding:** Domain alignment mattered more than fine-tuning
  The model pre-trained on movie reviews worked better than models fine-tuned on Reddit data
- **Main error pattern:** Ambiguous cases were the largest error type
  32% of analysed errors involved mixed or unclear sentiment
- **Takeaway:**
  Cross-domain sentiment analysis remains challenging when training and test data differ in style, length, and vocabulary